SLEEPING MACHINES • TUTORIAL 8

Formal Verification with Lean 4

Reading a theorem, checking its proof, and identifying what the formal statement does not claim

Why this tutorial

A numerical witness checks selected inputs. A proof assistant checks a statement for every input satisfying its explicit assumptions. Both are useful: a checked inequality does not verify an entire training program, and a successful model fit does not prove a scaling theorem. This tutorial teaches how to read the small formal cores supporting our credit theory.

1 Propositions, proofs and the kernel

Lean represents a proposition as a type and a proof as a term inhabiting that type. A tactic helps construct the term; a small kernel checks it. "The tactic succeeded" means the resulting term passed type checking, relative to its imported definitions and axioms. It says nothing about whether the formal proposition faithfully expresses the intended scientific claim.

Read hypotheses first. In this standalone example, h is a proof supplied as an input:

```
import Mathlib
example (x y : Real) (h : x <= y) : 2*x <= 2*y := by
  nlinarith
```

The proof does not show $x \le y$ universally. It shows that whenever this hypothesis holds, doubling preserves the inequality. Mathematical objects have types: `Real`, `Nat`, finite sets and finite sums cannot be interchanged without explicit casts.

2 Tactics as mathematical steps

- `intro` brings a quantified variable or implication hypothesis into the context; `exact` supplies a term with precisely the current goal's type.
- `rw` rewrites using an equality; `simp` applies a controlled collection of simplification lemmas.
- `ring` normalizes polynomial identities. It does not prove an analytic theorem about an integral or logarithm.
- `field_simp` clears denominators. Nonzero-denominator obligations must still be justified.
- `linarith` combines linear equalities/inequalities; `nlinarith` also preprocesses polynomial facts.
- `positivity` proves suitable sign facts from known hypotheses and the structure of an expression.

Example 1 (A denominator obligation)**.**
```
import Mathlib
example (x : Real) (hx : 0 < x) : x / x = 1 := by
  exact div_self (ne_of_gt hx)
```
The positive-denominator hypothesis supplies $x \neq 0$. For $x = 0$, Lean's field division gives $0/0 = 0$, so the unrestricted statement would be false.

3 Walk through the project

The project is `experiments/lean/credit_theory`. Its `lean-toolchain` and Lake manifest pin Lean and Mathlib. The current pin is Lean 4.35.0-rc4 with the matching Mathlib revision. Use the pinned project environment rather than silently upgrading it.

Basic.lean. `var_eq_half_pairwise` proves the finite weighted-variance identity given weights summing to one. The algebraic identity does not itself require nonnegative weights; `sampled_credit_variance_bound` adds nonnegativity and bounded scores to obtain the probability interpretation and bound. `geom_tail_le` proves a finite transported-tail bound for $0 \le \rho < 1$. `info_from_waiting` proves a logarithmic ratio equals $2r\tau$; identifying that ratio with complex Gaussian information is a separate statistical argument.

Scaling.lean. `count_above_level_le` bounds the number of components above a threshold by total mass divided by the threshold. `powerlaw_rate_le` bounds a finite logarithmic sum; `powerlaw_tail_le` needs $\alpha > 1$. These are implications from a spectrum model, not proofs that our measured spectra have that model.

Coupled.lean. `coupled_contraction` takes nonnegative comparison coefficients, $a, d < 1$ and $bc < (1-a)(1-d)$, then constructs $w > 0$ and $q \in [0,1)$ so $x' + we' \le q(x + we)$. Its finite-dimensional proof does not infer those coefficients from neural training. `coupled_converges` turns a supplied one-step potential bound into a geometric bound by induction.

Surprise.lean. `audit_cost_lower_bound` applies Cauchy–Schwarz to show $(\sum_i |r_i|\sqrt{c_i})^2/V \le \sum_i c_i p_i$, assuming positive p_i, V and $\sum_i r_i^2/p_i \le V$. The actual residual-audit variance is $\sum_i (1/p_i - 1) r_i^2$: the formal theorem uses the larger uncentered budget $\sum_i r_i^2/p_i$. Keeping these denominators distinct avoids claiming the wrong constant.

RaceFisher.lean. `race_fisher_offdiag` and `race_fisher_diag` check rational identities after the exponential-race moments have been supplied. They do not derive the moments, establish martingale integrability, or prove a shared neural parameter Fisher is diagonal.

4 Reproduce and inspect

With Elan and the pinned toolchain installed, run from the project directory:

```
lake exe cache get
lake build
```

The cache command obtains Mathlib artifacts; the build checks the project. Dependencies may require network access and substantial disk space. The tutorial PDF build is separate and does not establish a fresh Lean build on a host lacking this toolchain.

For an axiom inspection, save this as `Inspect.lean` in a temporary copy of the project:

```
import CreditTheory
#check coupled_contraction
#print axioms coupled_contraction
#print axioms sampled_credit_variance_bound
#print axioms race_fisher_diag
```

Then run `lake env lean Inspect.lean`. `#check` reports the type; `#print axioms` lists transitive axiom dependencies. Standard classical developments may use `propext`, `Classical.choice` and

`Quot.sound`. A dependency on `sorryAx` signals an admitted hole. An arbitrary declared axiom can also hide an unproved result even when there is no literal `sorry` in the theorem body.

Key idea

Review the statement, imported axioms, and connection to the implementation. Kernel acceptance checks logical derivation. It does not certify the data protocol, PyTorch gradients, clockless hardware, or superiority over BPTT.

Exercises

Exercise 1. Why is a theorem with hypothesis `h : False` unhelpful for model stability?

Solution. From falsehood any proposition follows. Unless the assumptions are satisfiable in the scientific setting, the checked implication carries no useful stability guarantee.

Exercise 2. An audit theorem assumes $p_i > 0$. Can deterministic omission of a block use it?

Solution. No: omission has $p_i = 0$ and removes residual-correction support. Prove a separate approximation bound or restore positive inclusion probability.

Exercise 3. Which extra checks connect `race_fisher_diag` to a training method?

Solution. Verify the observation law and stated moments, account for censoring and hidden causes, map log-scales to shared weights, and check the implementation's score/gradient. The rational identity supplies only one link in that chain.

Where this appears in our work

The Lean project accompanies notes 164–168. Numerical contracts separately check native temporal transport, state carry, full parameter gradients and changing-weight versions. Experiment result files supply quality and cost evidence. The Gaussian coding, renewal and neural calibration assumptions must be justified outside the finite algebra proved here.

Further reading. Theorem Proving in Lean 4, chapters on propositions, tactics and axioms; Lean's axiom reference. Project commands use the pinned revision above.